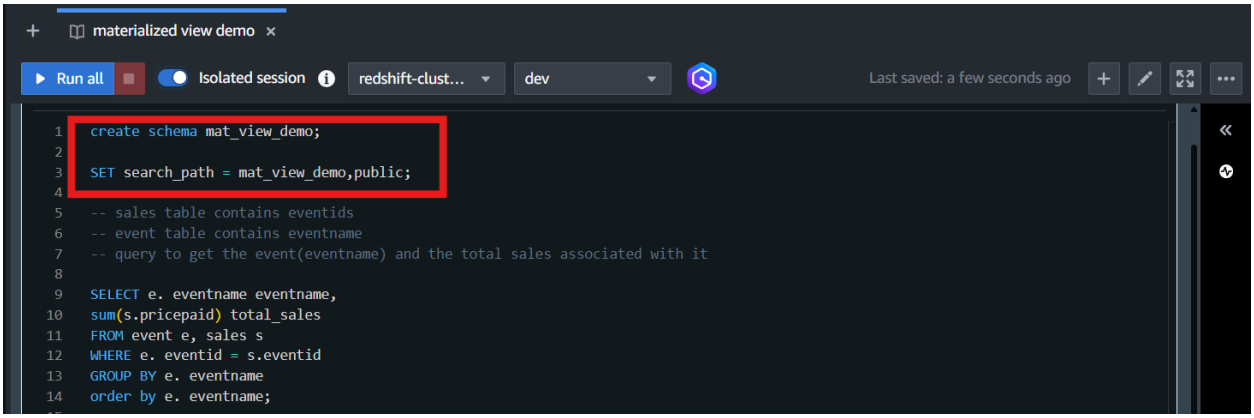
Materialized View in Amazon Redshift

To Begin with the Lab

Summary of the Lab

In this lab, a Materialized View named tickets_mv was created in **Amazon Redshift** using sales and event data from the Tickit sample database. A query joining the sales and event tables was executed first to analyze event-wise total sales. The query was then converted into a Materialized View to cache results and improve analytics performance for repetitive queries and dashboards. Base table data was later updated to demonstrate that Materialized Views use cached data until explicitly refreshed. Finally, the Materialized View was refreshed manually and validated successfully by confirming that the updated sales values were reflected in the query output.

- Navigate to **Amazon Redshift** → open **Redshift Query Editor v2**
- Create a schema for the lab data
- Connect to the Redshift cluster and select the dev database
- Create a new query editor tab for the lab

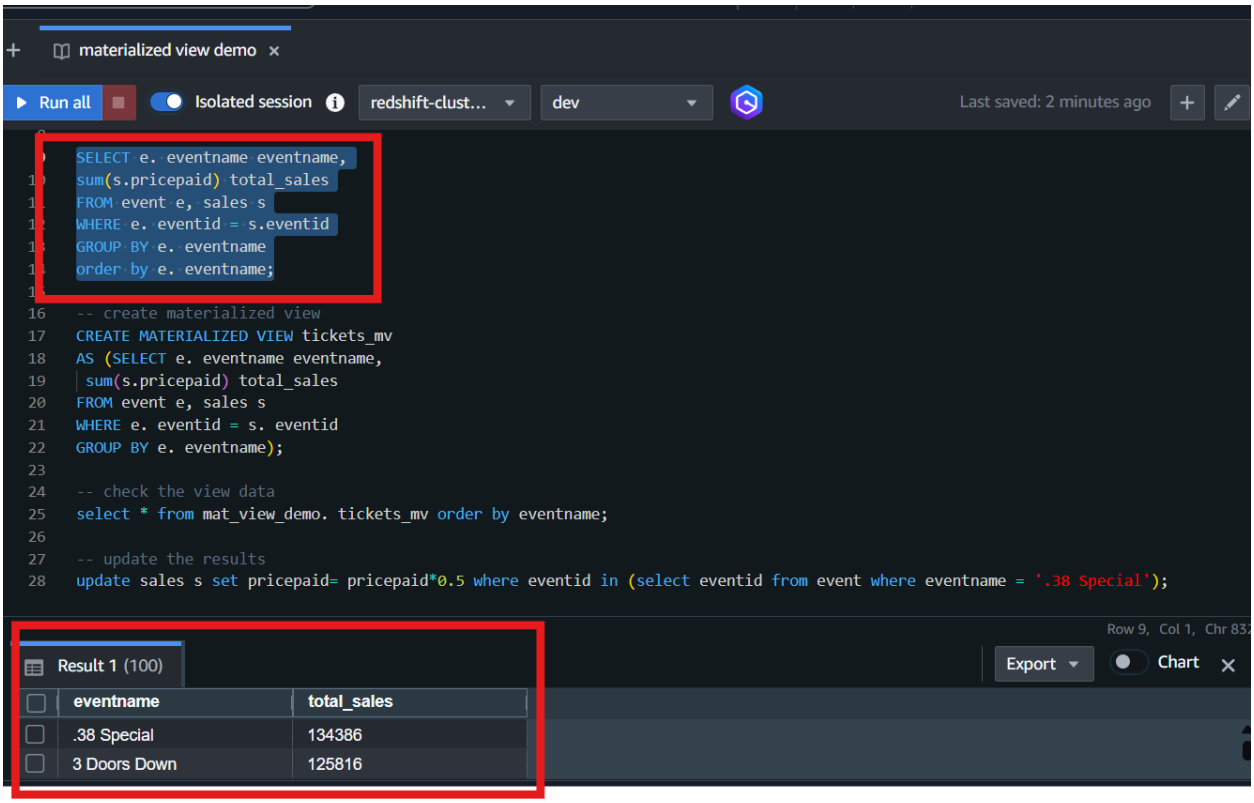


CREATE SCHEMA mat_view_demo;

- Set the schema search path

SET search_path TO mat_view_demo, public;

- Run the base query to calculate event-wise total sales



SELECT e.eventname eventname,

SUM(s.pricepaid) total_sales

FROM event e, sales s

WHERE e.eventid = s.eventid

GROUP BY e.eventname

ORDER BY e.eventname;

- Verify that the query returns 100 rows
- Create the Materialized View using the same query

CREATE MATERIALIZED VIEW tickets_mv AS

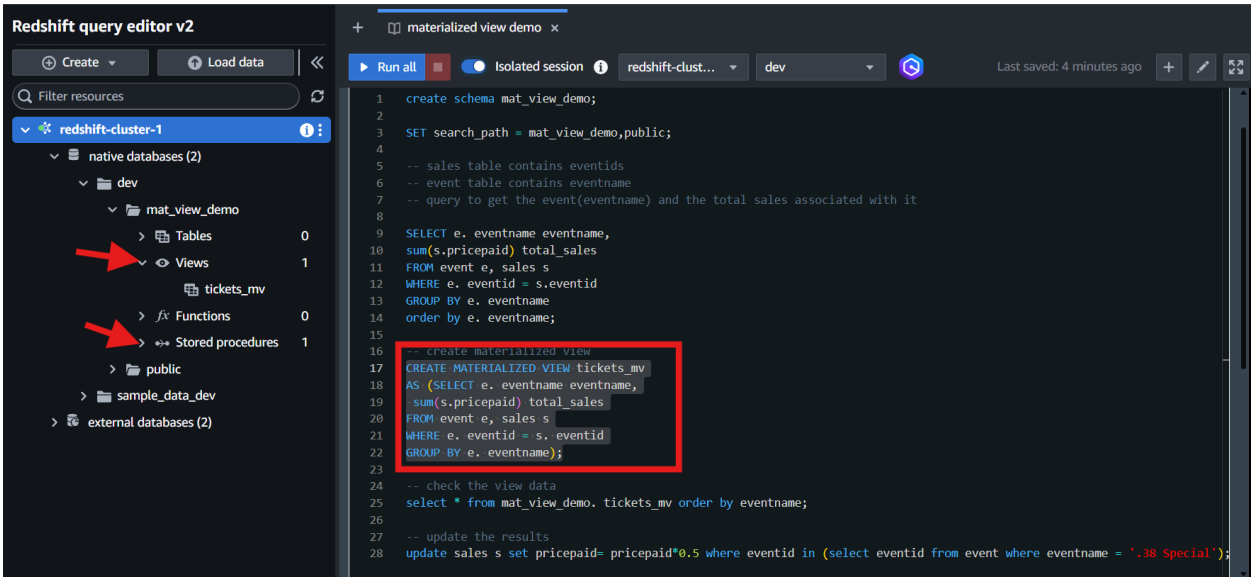
SELECT e.eventname eventname,

SUM(s.pricepaid) total_sales

FROM event e, sales s

WHERE e.eventid = s.eventid

GROUP BY e.eventname;

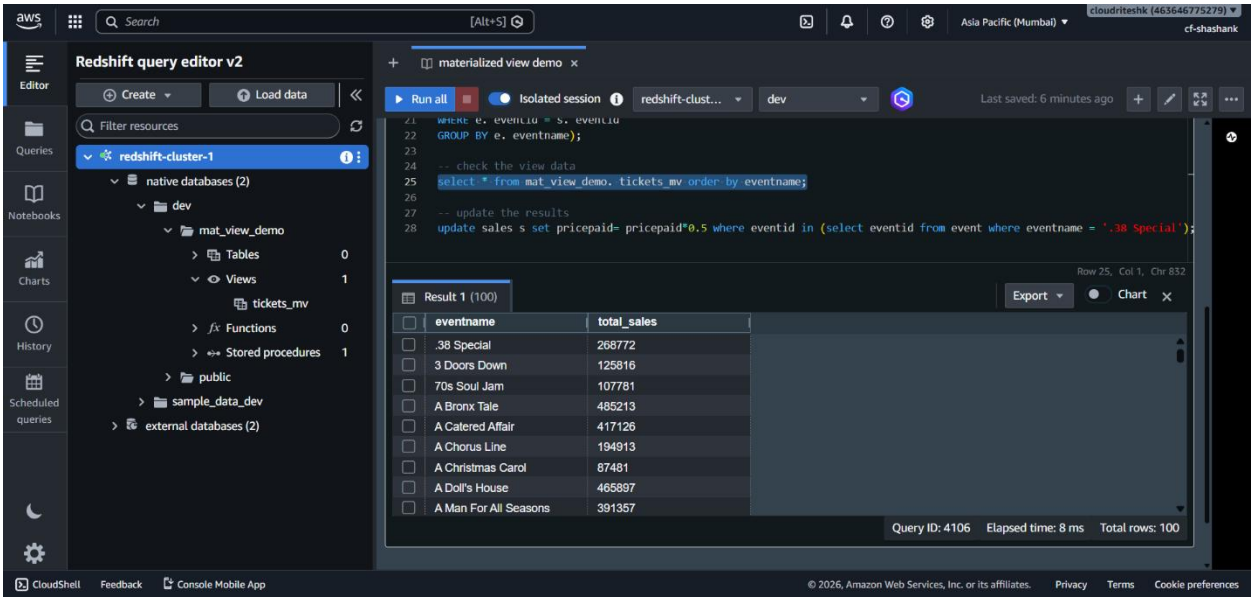


- Refresh the left navigation panel inside **Redshift Query Editor v2**
- Verify that the mat_view_demo schema now contains:
    - One View
    - One Internal Table
- Query the Materialized View

SELECT *

FROM mat_view_demo.tickets_mv

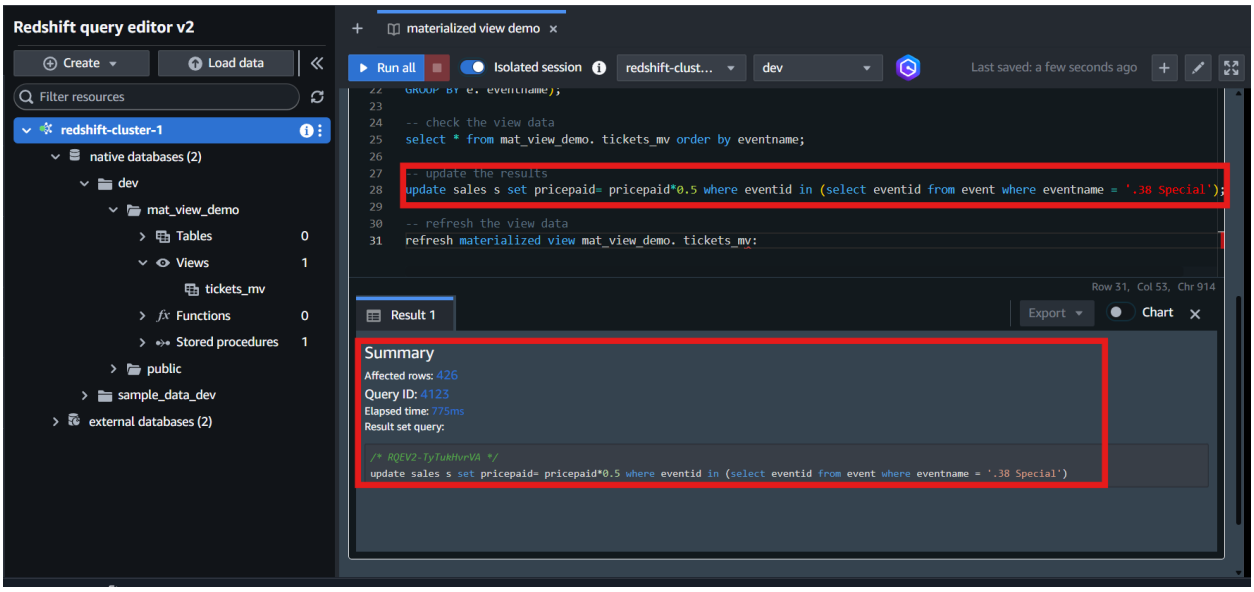
ORDER BY eventname;



- Verify that the Materialized View returns the same 100 rows as the original query
- Update the sales table data for the 0.38 Special event

```
UPDATE sales
SET pricepaid = pricepaid * 0.5
WHERE eventid IN (
  SELECT eventid
  FROM event
```

WHERE eventname = '0.38 Special');

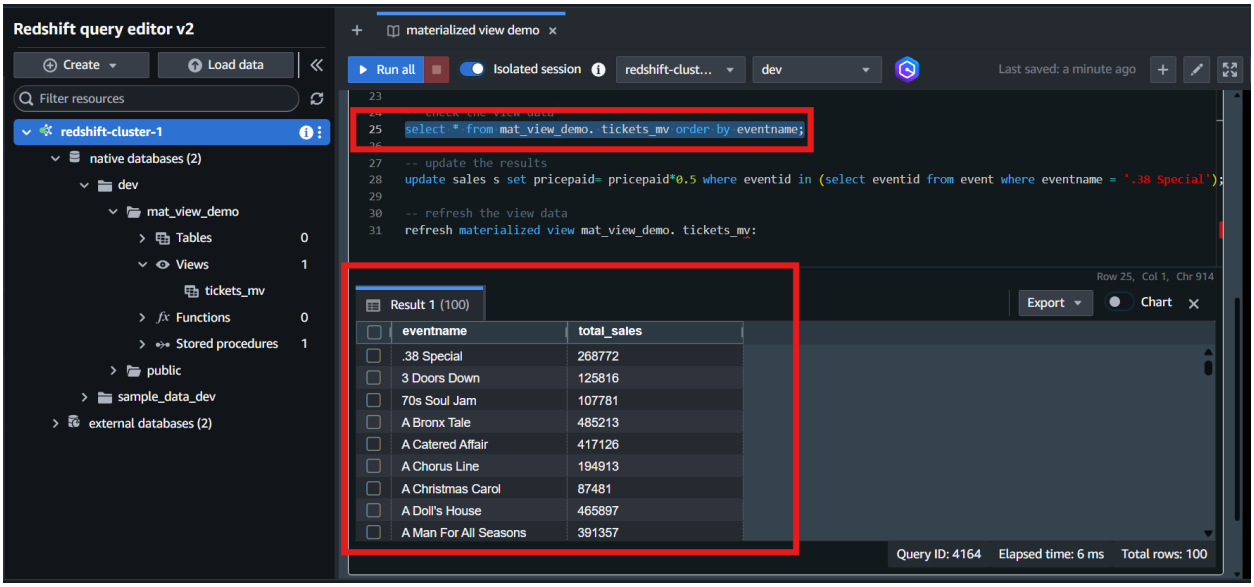


- Verify that approximately 426 rows were updated
- Query the Materialized View again

SELECT *

FROM mat_view_demo.tickets_mv

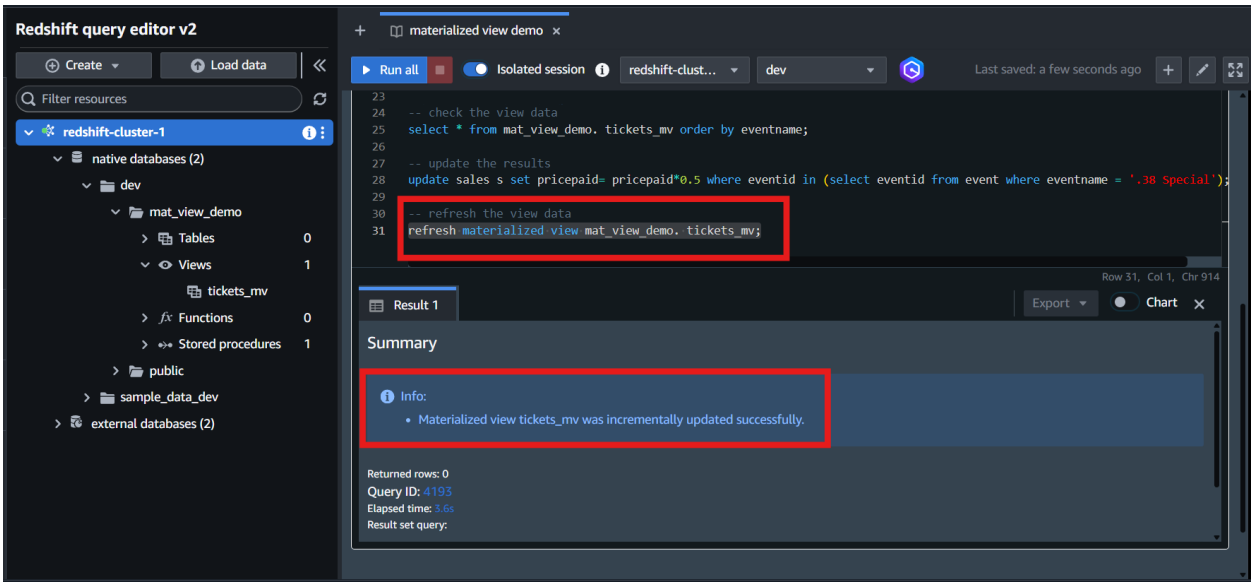
ORDER BY eventname;



- Observe that old cached values are still displayed

- Refresh the Materialized View manually

REFRESH MATERIALIZED VIEW mat_view_demo.tickets_mv;

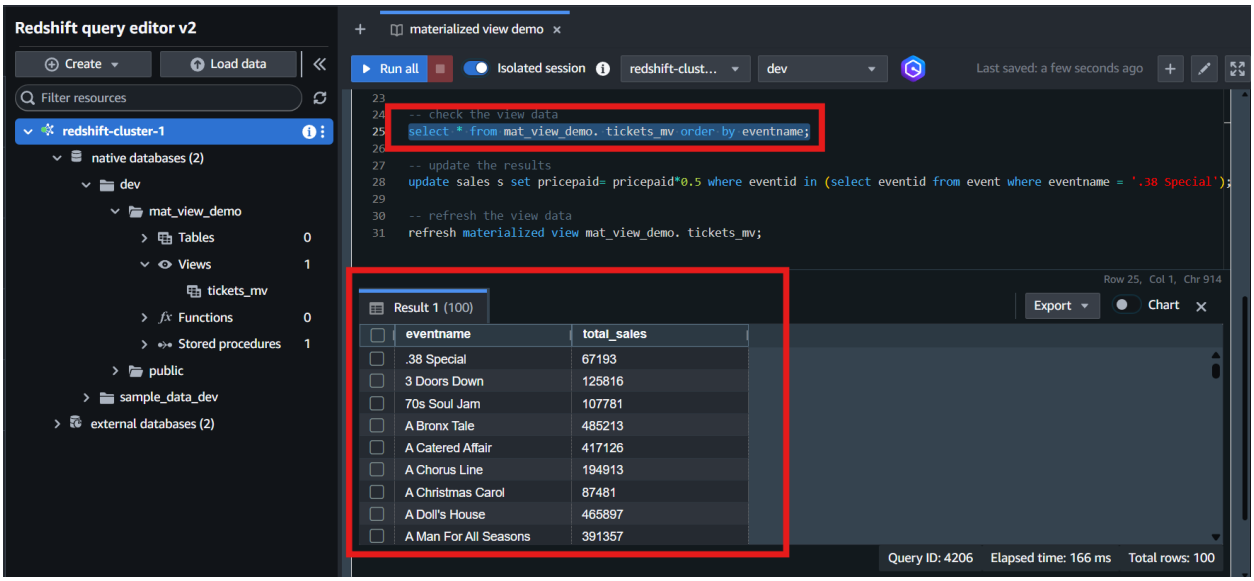


- Query the Materialized View after refresh

SELECT *

FROM mat_view_demo.tickets_mv

ORDER BY eventname;



- Verify that the 0.38 Special event now shows updated sales values of approximately 0.5+ million

- Confirm that the Materialized View refresh completed successfully and reflected the latest base table data